

پروژه پیاده‌سازی بازی شطرنج

عنوان درس: برنامه‌سازی پیشرفته – استاد خراشادی زاده بهار 1405

دانشجویان حل تمرین : علیرضا رحیم پناه - سهیل سیروسی

تاریخ تحویل: 1405/4/17

نوع پروژه: انفرادی / گروهی حداکثر ۲ نفره



۱. معرفی و هدف پروژه

در این پروژه، دانشجویان باید بازی رومیزی شطرنج (Chess) را با استفاده از زبان برنامه‌نویسی C++ پیاده‌سازی کنند. این بازی صرفاً به صورت دو نفره (Human vs. Human) طراحی شده و هدف اصلی آن، آشنایی عملی با مفاهیم زیر است:

برنامه‌نویسی شی‌گرا (OOP): شامل کلاس‌ها، وراثت، چندریختی (Polymorphism)، انتزاع (Abstraction) و کپسوله‌سازی (Encapsulation)

مدیریت حافظه (Memory Management): استفاده صحیح از اشاره‌گرها و جلوگیری از نشتی حافظه (Memory Leak)

طراحی ماژولار (Modular Design): جداسازی لایه‌های مختلف برنامه (منطق، نمایش، ورودی/خروجی).

تذکر: استفاده از هرگونه موتور هوش مصنوعی مانند الگوریتم Minimax یا کتابخانه‌های آماده شطرنج در این پروژه مجاز نیست و فقط پیاده‌سازی دستی منطق بازی مورد قبول خواهد بود.

۲. قوانین و منطق بازی (Game Rules)

شما موظف به پیاده‌سازی کامل و دقیق قوانین استاندارد فدراسیون جهانی شطرنج هستید. موارد زیر الزامی هستند:

۲-۱. صفحه بازی (Board)

صفحه شامل ۸ ستون و ۸ سطر (سطرهای ۱ تا ۸) است. ستون‌های a تا h و سطرهای ۱ تا ۸ است.

خانه‌ها به صورت متناوب روشن و تیره هستند. برای نمایش در کنسول لزومی به رنگ‌آمیزی نیست، اما نمایش مختصات الزامی است.

۲-۲. مهره‌ها (Pieces)

شش نوع مهره به شرح زیر باید پیاده‌سازی شوند:

نام مهره (فارسی) نام مهره (انگلیسی) نماد پیشنهادی برای کنسول سفید/سیاه

King شاه K / k

Queen وزیر Q / q

Rook رخ R / r

Bishop فیل B / b

Knight اسب N / n

Pawn سرباز P / p

۲-۳. حرکتهای ویژه اجباری

قلعه رفتن (Castling): هم به سمت راست (قلعه کوتاه) و هم به سمت چپ (قلعه بلند). شرایط انجام آن (حرکت نکردن شاه و رخ، خالی بودن مسیر، نبودن در کیش) باید رعایت شود.

آن پاسان (En Passant) در صورتی که سرباز حریف دو خانه جلو بیاید و در کنار سرباز شما قرار گیرد، باید امکان زدن آن به صورت آن پاسان وجود داشته باشد.

تبدیل سرباز (Pawn Promotion): زمانی که سرباز به انتهای صفحه (سطر ۸ برای سفید و سطر ۱ برای سیاه) می‌رسد، کاربر باید بتواند از بین مهره‌های (وزیر، رخ، فیل، اسب) یکی را انتخاب کرده و سرباز را به آن تبدیل کند.

۲-۴. تشخیص وضعیت‌های بازی

کیش (Check): برنامه باید پس از هر حرکت، وضعیت کیش بودن شاه حریف را تشخیص داده و به کاربر اعلام کند.

کیش و مات (Checkmate) و پات (Stalemate): در صورت بروز هر یک از این حالات، برنامه باید بازی را تمام کرده و بازیکن برنده یا حالت مساوی را اعلام کند.

۳. محدودیت‌ها و الزامات فنی

زبان برنامه‌نویسی C++ : استاندارد C++17 یا بالاتر - در صورت استفاده از ویژگی‌های جدید حتماً در README ذکر شود.

کتابخانه‌ها: استفاده از کتابخانه‌های استاندارد (STL) مانند `<vector>`، `<string>`، `<iostream>` و `<fstream>` بلامانع است. استفاده از کتابخانه‌های خارجی فقط برای بخش رابط گرافیکی (اختیاری) مجاز خواهد بود

۴. معماری و طراحی نرم‌افزار

برای دریافت نمره کامل در بخش طراحی، کد شما باید ماژولار بوده و از اصول SOLID پیروی کند. حداقل کلاس‌های زیر باید در پروژه تعریف شوند:

۴-۱. کلاس‌های اجباری

کلاس Piece پایه انتزاعی: شامل توابع مجازی خالص مانند isValidMove و getSymbol سایر مهره‌ها Pawn, Knight, Bishop, Queen, King Rook, باید از این کلاس ارث‌بری کنند.

کلاس Board: مسئولیت نگهداری آرایه‌ای از مهره‌ها (یا اشاره‌گر به آن‌ها) و اعمال تغییرات روی صفحه را بر عهده دارد.

کلاس Game: منطق اصلی بازی شامل کنترل نوبت‌ها، دریافت ورودی از کاربر، فراخوانی توابع اعتبارسنجی و بررسی وضعیت پایان بازی.

کلاس Display یا Renderer: وظیفه نمایش صفحه و پیام‌ها به کاربر را بر عهده دارد. این کلاس باید کاملاً از Game جدا باشد تا بتوان به راحتی آن را با یک رابط گرافیکی جایگزین کرد.

۴-۲. مدیریت حافظه

استفاده از اشاره گرهای خام (Raw Pointers) با new و delete مجاز است، اما استفاده از اشاره گرهای هوشمند (Smart Pointers) مانند std::shared_ptr و std::weak_ptr امتیاز مثبت دارد.

تأکید: نشستی حافظه (Memory Leak) در پروژه پذیرفته نیست.

۵. نحوه ورودی، خروجی و نمایش

۵-۱. دریافت حرکت از کاربر

حرکت باید به صورت مختصات مبدأ و مقصد دریافت شود. قالب استاندارد: [ستون مبدأ][سطر مبدأ][ستون مقصد][سطر مقصد]

مثال d2 d4 :

```
r\c  a b c d e f g h
-----
8 |  Rb Nb Bb Qb Kb Bb Nb Rb
7 |  Pb Pb Pb Pb Pb Pb Pb Pb
6 |  0 0 0 0 0 0 0 0
5 |  0 0 0 0 0 0 0 0
4 |  0 0 0 0 0 0 0 0
3 |  0 0 0 0 0 0 0 0
2 |  Pw Pw Pw Pw Pw Pw Pw Pw
1 |  Rw Nw Bw Qw Kw Bw Nw Rw

*****White's Turn*****
Enter the position from which you want to move the piece : d2d4
```

```
r\c  a b c d e f g h
-----
8 |  Rb Nb Bb Qb Kb Bb Nb Rb
7 |  Pb Pb Pb Pb Pb Pb Pb Pb
6 |  0 0 0 0 0 0 0 0
5 |  0 0 0 0 0 0 0 0
4 |  0 0 0 Pw 0 0 0 0
3 |  0 0 0 0 0 0 0 0
2 |  Pw Pw Pw 0 Pw Pw Pw Pw
1 |  Rw Nw Bw Qw Kw Bw Nw Rw

1. 1 (d 2) --> (d 4)
*****Black's Turn*****
Enter the position from which you want to move the piece : 
```

برنامه باید نسبت به حروف بزرگ و کوچک غیر حساس باشد یعنی هم D2 و هم d2 قابل قبول باشد.

۵-۲. نمایش صفحه در کنسول - خروجی اصلی

صفحه باید به صورت یک جدول مرتب در خروجی کنسول چاپ شود. مختصات ستون‌ها در بالا و مختصات سطرها در کنار جدول درج شوند.

نمونه خروجی پیشنهادی: * در مثال صفحه گذشته خروجی صفحه قابل مشاهده است

۶. قابلیت‌های اختیاری (ویژه نمره اضافی)

پیاده‌سازی هر یک از موارد زیر تا سقف 30 نمره مازاد برا 100 به امتیاز نهایی اضافه خواهد کرد:

1 - پیاده‌سازی پروژه در گیت مرحله به مرحله و گیت هاب

حتماً باید `readme` داشته باشه و با فرمت استانداردش نوشته بشه در غیر این صورت نمره لحاظ نخواهد شد

[لینک دوره آموزش گیت و گیت هاب جادی \(رایگان\)](#)

2 - استفاده از کاراکترهای یونیکد

دانشجویان می‌توانند به جای حروف لاتین، از اموجی‌های شطرنج (مانند ♔، ♚، ♛، ♜، ♞ و ...) برای زیباتر شدن خروجی استفاده کنند.

تذکرات فنی: در صورت استفاده از این ویژگی، حتماً کد منبع را با فرمت UTF-8 ذخیره کنید. کاربران ویندوز برای نمایش صحیح باید دستور

`SetConsoleOutputCP(CP_UTF8);` را در ابتدای برنامه قرار دهند. همچنین به دلیل پهنای دو کاراکتری بودن این نمادها، برای

چیدمان جدول حتماً از `\t (Tab)` استفاده کنید.

3 - رابط کاربری گرافیکی: (GUI)

کتابخانه‌های زیادی برای استفاده وجود دارد مثال:

پنجره گرافیکی برای بازی با استفاده از کتابخانه‌های SFML، Qt یا مشابه آن.

امکان حرکت مهره‌ها با کلیک ماوس (Drag & Drop) یا کلیک دو مرحله‌ای.

تذکر: منطق بازی (Game) و (Board) باید کاملاً از کد GUI جدا باشد و GUI فقط یک لایه نمایشی جدید باشد.

4 - ذخیره‌سازی و بارگذاری در فایل: (File I/O)

امکان ذخیره وضعیت فعلی بازی در یک فایل متنی.

امکان بارگذاری یک بازی ذخیره‌شده از فایل و ادامه بازی از همان نقطه.

فرمت فایل باید در README توضیح داده شود.

5 - پیاده سازی ماژولار و در چند فایل

۷. نحوه تحویل پروژه (فرمت ارسال)

لطفاً پروژه خود را در قالب یک فایل فشرده Zip یا Rar شامل موارد زیر تحویل دهید:

کدهای منبع: (Source Code) تمام فایل‌های .cpp و .h. به همراه فایل‌های مربوط به GUI در صورت وجود.

فایل راهنما: (README) یک فایل متنی یا PDF شامل:

نحوه کامپایل و اجرای پروژه دستورات کامپایلر یا توضیح مربوط به IDE

توضیح مختصری درباره معماری و کلاس‌های طراحی شده.

در صورت استفاده از کتابخانه‌های خارجی، نحوه نصب و لینک کردن آنها.

۸ جدول نمره‌دهی (معیارهای ارزیابی)

بخش نمره از ۱۰۰ توضیحات

کارکرد صحیح (Core Functionality) ۲۰ پیاده‌سازی کامل و بدون باگ حرکت مهره‌ها، قوانین ویژه (قلعه، آن‌پاسان، تبدیل) و تشخیص کیش/مات/پات

طراحی شی‌گرا و ماژولار (OOP Design) ۲۰ استفاده صحیح از وراثت، چندریختی، کپسوله‌سازی، جداسازی لایه‌ها و رعایت اصول SOLID

مدیریت حافظه (Memory Management) ۱۵ عدم وجود نشتی حافظه، استفاده بهینه از اشاره‌گرها، new/delete یا Smart Pointers

مستندسازی و کیفیت کد (Documentation) ۵ وجود کامنت‌های مفید، کد تمیز و خوانا، ارائه README مناسب

دفاع شفاهی و آزمون مفاهیم (Oral Defense) ۴۰ سوالات شفاهی در جلسه دفاع درباره مدیریت حافظه، اشاره‌گرها، وراثت، چندریختی، و نحوه پیاده‌سازی بخش‌های مختلف پروژه

ویژگی‌های اختیاری (امتیاز مازاد) Extra Credit -

تا 30 نمره بسته به کیفیت پیاده‌سازی و تلاش دانشجو

۹. قوانین دفاع شفاهی

توجه :

در صورت تشخیص سرپرست حل تمرین مبنی بر عدم توانایی دانشجو در توضیح کد خود و نشان دادن درک صحیح از پروژه، ایشان می‌توانند به تشخیص خود، از دادن نمره به سایر بخش‌های پروژه (کارکرد، طراحی، مدیریت حافظه، مستندسازی و ویژگی‌های اختیاری) خودداری کنند و صرفاً نمره دفاع شفاهی را لحاظ نمایند.

به عبارت دیگر، اگر دانشجو نتواند ثابت کند که خودش کد را نوشته و درک کرده است، حتی اگر کد به درستی کار کند، امکان دارد نمره‌ای از بخش‌های عملی دریافت نکند.

دفاع گروهی: در صورت پروژه گروهی، هر عضو به صورت جداگانه ارزیابی می‌شود.

موفق و پیروز باشید